A JOURNEY INTO COMPUTING FOR ENGINEERS

CSE 4105

EDITED & COMPILED BY

ABRAR MAHMUD HASAN

“Why was the computer cold? Because it left its windows open”

Binary File formats

A binary file is a file whose content is in a binary format consisting of a series of sequential bytes, each of which is eight bits in length. The content must be interpreted by a program or a hardware processor that understands in advance exactly how that content is formatted and how to read the data. Binary files include a wide range of file types, including executables, libraries, graphics, databases, archives and many others. Each executable file has a common format called **Common Object File Format (COFF)**, a format for executable, object code, and shared library computer files used on Unix systems. For example, PE and ELF files.

ELF (Executable and Linkable format)

ELF is the standard binary format on operating systems such as Linux. Some of the capabilities of ELF are dynamic linking, dynamic loading, imposing run-time control on a program, and an improved method for creating shared libraries. The ELF representation of control data in an object file is platform independent, which is an additional improvement over previous binary formats.

The ELF representation permits object files to be identified, parsed, and interpreted similarly, making the ELF object files compatible across multiple platforms and architectures of different size. The three main types of ELF files are:

- Executable
- Relocatable
- Shared object

These file types hold the code, data, and information about the program that the operating system and linkage editor need to perform the appropriate actions on these files.

PE (Portable Executable)

The Portable Executable (PE) file format is the standard file format for executable files, object code, DLLs (Dynamic Link Libraries), and others in versions of Windows operating systems. The structure of a PE file consists of several components that organize and define various aspects of the executable. Here's an overview of the main parts of the PE file structure:

DOS Header (MZ Header): The DOS Header contains the DOS MZ executable signature ('MZ') and a small DOS program, often referred to as DOS stub or DOS stub program.

This header allows older versions of Windows to detect that the file is not a DOS executable.

PE Signature and COFF File Header: After the DOS Header, there's a signature indicating the start of the PE header ('PE\0\0'). Following the PE signature is the COFF (Common Object File Format) File Header, which contains information about the architecture, sections, timestamps, etc.

Optional Header: This section provides additional information about the executable. It includes details such as the preferred base address, entry point, image size, section alignment, and subsystem requirements (console, GUI, etc.). The Optional Header can be standard (for object files) or optional (for executable files).

Section Headers: The Section Headers describe various sections within the PE file, including .text (executable code), .data (initialized data), .rsrc (resources), etc. Each section header holds information like section names, virtual sizes, file offsets, etc.

Data Directories: The Optional Header includes a Data Directory array. Each entry in this array points to a specific directory containing important information such as imports, exports, resources, exception handling, debug information, and more.

Sections: These are segments that contain different types of data within the file, like code, data, resources, and more. Sections are aligned and contain raw data or executable code.

Import and Export Tables: The Import Table lists functions imported by the executable, specifying the libraries and functions used. It contains details about DLLs and functions that the executable requires. The Export Table contains information about functions exported by the executable, allowing other programs to use them.

Resource Section: This section holds resources such as images, icons, dialogs, version information, and other non-executable data used by the application.

Relocation Information: The Relocation Table contains information necessary for adjusting addresses during the loading process if the preferred load address isn't available.

Read more at: https://tech-zealots.com/malware-analysis/pe-portable-executable-structure-malware-analysis-part-2/

Cloud Computing

Cloud computing refers to the delivery of computing services—including servers, storage, databases, networking, software, analytics, and more—over the internet ("the cloud") to offer faster innovation, flexible resources, and cost savings. Instead of owning physical hardware and managing infrastructure, users can access these resources on-demand from a cloud service provider.

Key components and characteristics of cloud computing include:

1. **On-Demand Service**: Users can access computing resources as needed and pay only for what they use, often on a pay-as-you-go or subscription-based model.

2. **Scalability:** Cloud services can be easily scaled up or down based on demand, allowing businesses to efficiently handle fluctuating workloads without significant infrastructure changes.

3. **Resource Pooling:** Providers offer a multi-tenant model, where resources are shared among multiple users, optimizing resource utilization and providing economies of scale.

4**. Elasticity:** Cloud services can automatically scale resources to accommodate changing demands, ensuring optimal performance and avoiding downtime.

5. **Self-Service:** Users can provision and manage resources through a web interface or API without the need for extensive IT expertise or intervention.

6. **Broad Network Access:** Services are accessible over the internet from various devices, enabling users to access applications and data remotely.

7. **Measured Service:** Providers monitor and track usage metrics, allowing users to understand and optimize their resource consumption while providing transparency for billing.

Cloud computing offers several **deployment models:**

1. **Public Cloud:** Services are delivered over the internet and shared among multiple users on a pay-as-you-go basis. Examples include AWS (Amazon Web Services), Microsoft Azure, and Google Cloud Platform (GCP).

2. **Private Cloud:** Computing resources are dedicated to a single organization and are hosted on-premises or provided by a third-party vendor. They offer more control and security but require higher maintenance costs.

3. **Hybrid Cloud:** Combines elements of both public and private clouds, allowing data and applications to be shared between them. It provides flexibility and helps leverage the benefits of both models.

4. **Multi-Cloud:** Involves using services from multiple cloud providers to avoid vendor lock-in, improve redundancy, and optimize performance and cost.

Cloud computing services encompass a wide array of offerings provided by cloud service providers. These services are categorized into different models based on the type of resources they offer and how they are accessed. Some of the primary cloud computing services include:

1. **Infrastructure as a Service (IaaS)**: Provides virtualized computing resources over the internet, including virtual machines (VMs), storage, and networking. Examples: Amazon EC2, Microsoft Azure Virtual Machines, Google Compute Engine.

2. **Platform as a Service (PaaS):** Offers a platform allowing customers to develop, run, and manage applications without dealing with the underlying infrastructure. Examples: Heroku, Google App Engine, Microsoft Azure App Service.

3. **Software as a Service (SaaS):** Delivers software applications over the internet, eliminating the need for installation, maintenance, and management by the user. Examples: Google Workspace (formerly G Suite), Microsoft Office 365, Salesforce, Dropbox.

4. **Function as a Service (FaaS) / Serverless Computing**: Allows developers to run individual functions or application code without managing the infrastructure. Examples: AWS Lambda, Azure Functions, Google Cloud Functions.

5. **Database as a Service (DBaaS)**: Provides managed database services, offering features like automated backups, scaling, and maintenance. Examples: Amazon RDS, Azure SQL Database, Google Cloud SQL.

6. **Container as a Service (CaaS)**: Offers a platform for deploying, managing, and orchestrating containers (e.g., Docker) and containerized applications. Examples: Amazon ECS, Azure Kubernetes Service (AKS), Google Kubernetes Engine (GKE).

7. **Storage as a Service**: Provides scalable and accessible storage solutions over the internet. Examples: Amazon S3, Google Cloud Storage, Azure Blob Storage.

8. **Networking as a Service**: Offers services related to networking, such as virtual private networks (VPNs), content delivery networks (CDNs), and load balancing. Examples: AWS Direct Connect, Azure Virtual Network, Google Cloud CDN.

9. **AI/ML Services**: Cloud providers offer machine learning and artificial intelligence services, including pre-trained models, APIs for vision, language processing, etc. Examples: AWS AI/ML Services (Amazon Rekognition, Amazon Comprehend), Azure AI, Google Cloud AI.

These cloud computing services empower organizations to build, deploy, and manage applications and IT resources efficiently, providing scalability, flexibility, cost-effectiveness, and accessibility to a wide range of technologies without the need for substantial upfront investments in hardware or infrastructure.

Virtualization

Virtualization is the process of creating a virtual (rather than physical) version of something, such as hardware, storage devices, operating systems, or network resources. It allows multiple virtual instances or environments to run on a single physical hardware system. The primary goal of virtualization is to improve resource utilization, flexibility, and efficiency in computing.

A Virtual Machine (VM) is an emulation of a computer system that operates as an independent entity within a physical computer. VMs are created through virtualization, enabling multiple VMs to run on a single physical machine simultaneously.

Here are **key components** and aspects of virtualization and VMs:

Hypervisor: Also known as a Virtual Machine Monitor (VMM), the hypervisor is the core software that enables the creation and management of VMs. It controls and allocates physical hardware resources to VMs while isolating each VM from others.

Host Machine: The physical hardware or server that hosts and manages the VMs. It runs the hypervisor software and provides computing resources such as CPU, memory, storage, and networking to the VMs.

Guest Operating Systems: Each VM runs its own guest operating system, which operates independently from the host and other VMs. These guest OSs can be different from the host OS or from each other, depending on the virtualization setup.

Isolation and Resource Allocation: Virtualization allows for the isolation of VMs from one another. Each VM operates in its own encapsulated environment, ensuring that activities within one VM do not affect others. Resource allocation, such as CPU, memory, and storage, can be dynamically adjusted among VMs based on demand.

Types of Virtualization:

Full Virtualization: Here, the hypervisor simulates the complete hardware environment, allowing unmodified guest operating systems to run.

Para-Virtualization: Requires modifications to the guest OS to enhance performance and efficiency by allowing direct communication with the hypervisor.

Hardware-Assisted Virtualization: Utilizes specific CPU features (Intel VT-x, AMD-V) to improve virtualization performance.

Advantages of Virtualization:

- Increased efficiency by better resource utilization.
- Cost savings through server consolidation and reduced hardware requirements.
- Improved scalability and flexibility in deploying and managing resources.
- Enhanced disaster recovery and backup options.

Computing Paradigms

Computing paradigms refer to different models or approaches for performing computational tasks, each with its unique principles, methodologies, and characteristics. These paradigms have evolved over time to address various computing needs, advancing technology, and changing demands. Some major computing paradigms include:

Classical Computing: Traditional computing based on binary logic and the execution of instructions by a central processing unit (CPU). It operates on binary bits (0s and 1s) and uses algorithms to perform calculations and process data.

Distributed Computing: Involves the use of multiple interconnected computers or systems that work together to accomplish a common task. It focuses on sharing resources, coordinating actions, and solving problems that may be too large or complex for a single system.

Parallel Computing: Utilizes multiple processors or cores to simultaneously execute computations, speeding up processing tasks. It involves dividing tasks into smaller parts and processing them concurrently for efficiency gains.

Cloud Computing: Provides access to computing resources over the internet on a pay-as-you-go or subscription-based model. It enables users to access and use scalable and on-demand computing resources, such as servers, storage, databases, and applications, without owning the physical infrastructure.

Edge Computing: Focuses on processing data and performing computations closer to the data source or "edge" of the network, reducing latency and improving response times. It involves deploying computing resources and services closer to the location where data is generated or consumed.

Quantum Computing: Relies on the principles of quantum mechanics to perform computations using quantum bits (qubits). Unlike classical bits, qubits can exist in multiple states simultaneously, allowing quantum computers to solve certain problems exponentially faster than classical computers.

Grid Computing: Connects geographically dispersed resources to form a distributed infrastructure for large-scale computational tasks. It enables sharing computing power, storage, and other resources across multiple administrative domains.

Fog Computing: Extends cloud computing capabilities to the edge of the network, providing computation, storage, and networking services between end devices and the cloud. It aims to reduce bandwidth usage and improve efficiency by processing data locally.

Read more at: https://www.geeksforgeeks.org/different-computing-paradigms/

Computing paradigm shift

The evolution of computing has seen various stages, each introducing new paradigms, architectures, and approaches to computing. Here's an overview of the transition through different computing eras:

Mainframe Computing (1950s-1970s): Mainframes were the first dominant computing systems, characterized by large, centralized machines capable of handling extensive data processing tasks for organizations. They were expensive, required specialized expertise, and were primarily used by large corporations for critical business applications and data processing.

Personal Computing (1980s-1990s): The introduction of personal computers (PCs) brought computing power to individuals and smaller organizations. PCs were smaller, more affordable, and allowed users to perform tasks like word processing, spreadsheets, and basic computing activities.

Client-Server Architecture (1980s-2000s): Client-server architecture emerged, dividing computing tasks between "client" machines (end-user devices) and "server" machines (centralized systems providing services or data). This architecture facilitated distributed computing, allowing multiple clients to access shared resources on servers, leading to improved efficiency and scalability.

Web Applications (1990s-Present): The rise of the internet led to the development of web-based applications accessible via web browsers. Web applications allowed for easier distribution, accessibility, and sharing of data and services across various platforms and devices.

Cluster Computing (Late 1990s-Present): Cluster computing involves connecting multiple computers (nodes) to work together as a single system to achieve high-performance computing (HPC). Clusters use parallel processing to tackle complex tasks, often found in scientific research, simulations, and data-intensive applications.

Grid Computing (1990s-2010s): Grid computing expanded on cluster computing by connecting geographically distributed resources to form a unified infrastructure. Grids allowed sharing of computing power, storage, and other resources across organizational boundaries for large-scale tasks.

Utility Computing (2000s-Present): Utility computing introduced a pay-per-use model, similar to traditional utilities like electricity or water. Users pay for computing resources based on usage. It aimed to provide computing resources on-demand, offering scalability, flexibility, and cost savings.

Cloud Computing (2000s-Present): Cloud computing represents a shift towards delivering computing services over the internet on a scalable, pay-as-you-go basis. It offers a wide range of services (IaaS, PaaS, SaaS) and features like on-demand access, scalability, flexibility, and resource pooling.

Blockchain

Blockchain is a decentralized and distributed ledger technology which is used to record information, like transactions, entry and other use cases. As the name suggests, blockchain is a chain of blocks which we can consider analogous to a ledger book which is used to keep track of business transactions. In the same manner, a blockchain operates as a continuously growing list of records called blocks. These blocks are chained or linked together through various cryptographic means which is the basis of blockchain technology.

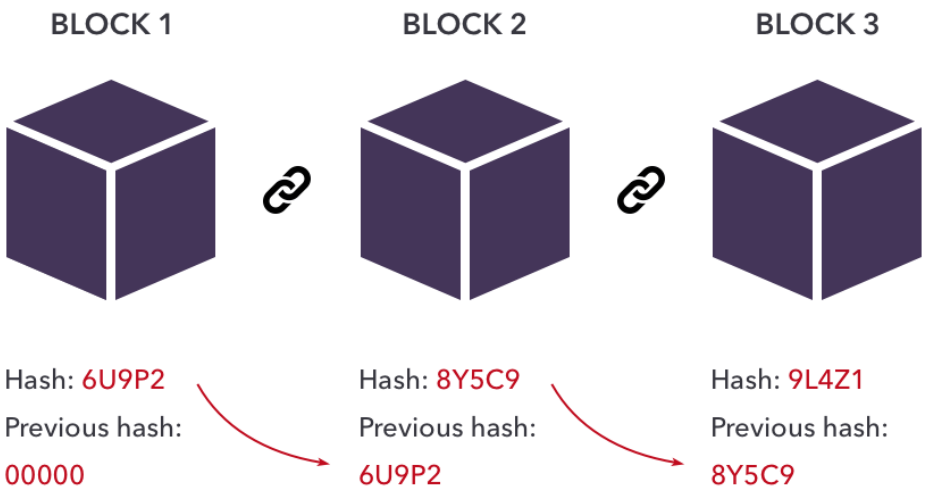

Figure: Simplified blockchain structure

Each block in a blockchain contains 3 items:

1. The data it is supposed to store
2. The hash of the block
3. The hash of the previous block

A general and simplified flow for **how blockchains work** is presented below:

- Transaction Initiation: When a transaction occurs, it is grouped together with other transactions into a block.
- Verification: Nodes on the network validate the transaction's authenticity by solving complex mathematical problems. This process is known as "proof of work" or "proof of stake", and it requires computational power or ownership of a certain amount of cryptocurrency to validate transactions.
- Block Formation: Once verified, the transaction is combined with other validated transactions to form a block. Each block contains a unique cryptographic hash (a digital fingerprint) of the previous block, creating a chain of blocks—hence the name "blockchain."

- Consensus: Consensus mechanisms ensure that all nodes agree on the validity of transactions and the order in which they are added to the blockchain. This agreement prevents double-spending and fraud.
- Decentralization: The blockchain is distributed across a network of computers, making it decentralized. Each node maintains a copy of the entire blockchain, ensuring transparency and security. This type of process is known as Peer-to-Peer(P2P)
- Immutability: Once a block is added to the blockchain, it is nearly impossible to alter or delete the information it contains due to the cryptographic hashes linking the blocks and the distributed nature of the network.
- Updating the Ledger: As new transactions occur, they are added to a new block and linked to the previous blocks, continuously extending the blockchain.

In the context of blockchain, several **terms and concepts** play crucial roles in ensuring the security, integrity, and functionality of the network. Here's an overview of each:

Transaction Validation: Transaction validation in blockchain involves confirming the legitimacy and authenticity of transactions before adding them to a block. It typically includes verifying the digital signatures, ensuring the sender has sufficient funds, and checking the transaction conforms to the network's rules (consensus rules).

Wallet: A wallet in the blockchain context is a software application or service that allows users to store, manage, and interact with their cryptocurrency assets. It generates and stores public and private keys, enabling users to send and receive digital currencies.

Half Node and Full Node:

Half Node: Also known as a "light node," it stores a partial copy of the blockchain, validating transactions but not keeping a complete record of all transactions. It relies on full nodes for transaction verification.

Full Node: A full node stores the entire blockchain ledger locally, verifies and validates transactions, and participates in the consensus process by maintaining the network's security and integrity.

Byzantine Fault Tolerance (BFT): Byzantine Fault Tolerance refers to a system's ability to function effectively and reach consensus even if some nodes or actors within the network are acting maliciously or failing to respond accurately. It ensures the network remains secure and operational despite potential faults or attacks.

Hash Tree (Merkle Tree): A Hash Tree or Merkle Tree is a data structure used in blockchain to efficiently verify the integrity of large sets of data. It organizes transaction data into a tree structure of hashed pairs, where each leaf node represents a transaction and each non-leaf node is the hash of its children. This structure allows for quick verification of large data sets by confirming the authenticity of smaller hashes, reducing computational load and ensuring data integrity.

Web Vulnerabilites

Sniffing (Network Sniffing): Network sniffing involves intercepting and capturing data packets transmitted over a network. Attackers can use specialized tools to monitor network traffic, potentially accessing sensitive information like login credentials, personal data, or financial details transmitted in plain text.

Man-in-the-Middle (MITM) Attack: MITM attacks occur when an attacker intercepts and potentially alters communications between two parties without their knowledge. This allows the attacker to eavesdrop on sensitive information or modify data being exchanged, leading to potential data compromise.

Phishing: Phishing is a social engineering attack where attackers use deceptive emails, messages, or websites to trick individuals into revealing sensitive information such as login credentials, financial details, or personal information. Phishing attacks often imitate legitimate entities to gain trust and deceive users.

SQL Injection (SQLi): SQL injection occurs when attackers insert malicious SQL code into input fields or data entry points of a web application. If the application does not properly sanitize or validate the input, attackers can manipulate SQL queries, potentially gaining unauthorized access to databases, extracting data, or performing other malicious actions.

Buffer Overflow: Buffer overflow vulnerabilities occur when a program attempts to write more data to a buffer (memory storage) than it can hold, leading to overwriting adjacent memory areas. Attackers can exploit this vulnerability to inject and execute malicious code, potentially gaining control of the system or causing crashes.

Null Pointer Exception: A Null Pointer Exception occurs in programming when a program attempts to access or manipulate data stored at a null (non-existent) memory address. Attackers may attempt to exploit this vulnerability to cause application crashes or execute arbitrary code.

Cross-Site Scripting (XSS): XSS vulnerabilities allow attackers to inject malicious scripts into web pages viewed by other users. These scripts execute within the victim's browser context, enabling the attacker to steal cookies, session tokens, or other sensitive information, deface websites, or perform other malicious actions.

UNIX

UNIX is a family of multitasking, multiuser computer operating systems originally developed in the 1960s and 1970s. It is known for its robustness, stability, and flexibility. Here are some key aspects of UNIX:

History and Origin: UNIX was developed at Bell Labs (AT&T) in the late 1960s by Ken Thompson, Dennis Ritchie, and others. It was designed to be portable, allowing it to run on various hardware architectures.

Features:

- **Multitasking:** UNIX allows multiple processes to run concurrently, enabling multitasking capabilities.
- **Multiuser:** It supports multiple users accessing the system simultaneously, each with their own login and permissions.
- **Hierarchical File System:** UNIX uses a hierarchical directory structure, organizing files in a tree-like format.
- **Shell:** The UNIX shell provides a command-line interface for users to interact with the operating system, execute commands, and manage files and processes.
- **Portability:** UNIX was designed with portability in mind, allowing it to be adapted and used across various hardware platforms.
- **Networking:** UNIX systems have robust networking capabilities, contributing to their widespread use in server environments and the internet's early development.

Variants and Distributions: Over time, various UNIX-like operating systems and distributions emerged. Notable variants include:

- **Linux:** Based on the UNIX principles, it is open-source and comes in various distributions (e.g., Ubuntu, Fedora, CentOS).
- **BSD (Berkeley Software Distribution):** A UNIX derivative developed at the University of California, Berkeley.
- **macOS:** Apple's operating system is built on a UNIX-based foundation (Darwin) with a graphical user interface (GUI).

Applications: UNIX and its variants are widely used in server environments, scientific research, academia, and enterprise settings due to their stability, security, and support for networking.

Standards and Compatibility: Various standards and specifications have been developed to ensure compatibility among UNIX-like systems, such as POSIX (Portable Operating System Interface for UNIX).

Security and Permissions: UNIX systems have robust security features, including user permissions and access control, helping administrators manage and control access to files and resources.

Linux Shell Commands

Navigation Commands:

cd: Change directory. Usage: cd directory_name
pwd: Print working directory. Shows the current directory path.
ls: List directory contents. Usage: ls [options] [directory]
mkdir: Create a new directory. Usage: mkdir directory_name
rmdir: Remove directory. Usage: rmdir directory_name

File Operations:

touch: Create an empty file or update file timestamps. Usage: touch file_name
cp: Copy files/directories. Usage: cp source_file destination_file
mv: Move/rename files/directories. Usage: mv source_file destination_file
rm: Remove/delete files/directories. Usage: rm file_name

Viewing and Editing Files:

cat: Display file content. Usage: cat file_name
less or more: View files page-by-page. Usage: less file_name
nano or vi or vim: Text editors for creating or modifying files. Usage: nano file_name, vi file_name

File Permissions:

chmod: Change file permissions. Usage: chmod permissions file_name
chown: Change file owner and group. Usage: chown owner:group file_name

Process Management:

ps: Display currently running processes. Usage: ps [options]
kill: Terminate a process by its process ID (PID). Usage: kill PID

System Information:

uname: Display system information. Usage: uname [options]
df: Show disk space usage. Usage: df [options]
free: Display system memory usage. Usage: free [options]

Networking:

ping: Check connectivity to a server or network device. Usage: ping server_name
ifconfig or ip: Display network interface information. Usage: ifconfig, ip addr show

Archiving and Compression:

tar: Create or extract tar archives. Usage: tar [options] archive_name
gzip or gunzip: Compress or decompress files. Usage: gzip file_name, gunzip file_name.gz

These commands provide a basic overview of Linux shell commands. Each command has various options and arguments that can be used for specific functionalities and tasks. Using the **man command** followed by the command name (e.g., man ls) provides access to the manual pages for detailed information about each command's usage and options.

File System

A file system is a method used by operating systems to manage and organize data on storage devices such as hard drives, SSDs (Solid State Drives), USB drives, and more. It provides a structure for storing, retrieving, and managing files and directories on storage media. Here are the key components and concepts related to file systems:

File Hierarchy:

A file system organizes data in a hierarchical structure, usually starting from a root directory (/) and branching out into subdirectories and files. This structure enables users to organize and access data efficiently.

Files and Directories:

Files: These are units of data storage containing information such as text, documents, programs, images, or any other type of information. Each file has a unique name and attributes.

Directories (Folders): Directories are containers used to organize files hierarchically. They can contain both files and other directories.

File System Types:

FAT (File Allocation Table): Used in older Windows systems, it's a simple file system supporting compatibility with various devices.

NTFS (New Technology File System): Widely used in modern Windows operating systems, offering improved features like file-level security, compression, and larger file size support.

EXT (Extended File System): Commonly found in Linux distributions, with variants like ext2, ext3, and ext4, offering features like journaling, reliability, and scalability.

APFS (Apple File System): Developed by Apple, used in macOS, iOS, and other Apple devices, offering features like encryption, snapshots, and improved performance.

HFS+ (Hierarchical File System Plus): An older file system used in macOS before APFS.

File System Operations:

File Creation and Deletion: Creating new files or deleting existing files.

File Modification: Editing, appending, or changing the contents of files.

File Access and Permissions: Controlling access to files through permissions (read, write, execute) for users and groups.

File System Maintenance: Activities like formatting, defragmentation, and checking for errors on storage devices.

File System Metadata: File systems store metadata information about files and directories, including attributes like file names, sizes, timestamps (creation, modification), ownership, and permissions.

File System Security: File systems provide security mechanisms to protect data, including file permissions, access controls, encryption, and user authentication.

Journaling and Recovery: Some modern file systems use journaling to record changes before writing them to disk, improving reliability and aiding in quick recovery in case of system crashes or power failures.

Hardware related topics

Computer Bootup:

Bootup is the process by which a computer starts and loads its operating system (OS) into memory, making it ready for user interaction. The bootup process typically involves the following stages:

Power On Self Test (POST): Hardware components like CPU, memory, and peripherals are tested to ensure they are functioning correctly.

BIOS/UEFI Initialization: The Basic Input/Output System (BIOS) or Unified Extensible Firmware Interface (UEFI) initializes hardware and performs system checks before loading the OS.

Boot Loader: The boot loader (e.g., GRUB for Linux, NTLDR for Windows) is loaded, which locates and loads the operating system kernel into memory.

Operating System Initialization: Once the kernel is loaded, the operating system starts initializing hardware, launching system services, and presenting the user interface.

Scheduler (Task Scheduler):

The scheduler is a component of an operating system responsible for managing and prioritizing the execution of various tasks or processes on the CPU. It determines which processes or threads should run, for how long, and in what order, based on scheduling algorithms. Schedulers ensure efficient CPU utilization, prevent resource contention, and maintain system responsiveness by managing the execution of tasks.

Timer:

A timer is a hardware or software mechanism used to measure and control time intervals in a computer system. It can be implemented through hardware timers (built into the CPU or motherboard) or software timers managed by the operating system. Timers are essential for various purposes, including scheduling tasks, triggering events, measuring time intervals for performance monitoring, and implementing time-based operations.

Cron expressions

Cron expressions are used in Unix-based operating systems to schedule tasks and automate recurring jobs. They define specific time intervals or patterns for executing commands or scripts at predetermined times. The cron system relies on the cron daemon, a background process that runs scheduled tasks based on these expressions.

A typical cron expression consists of five fields representing time elements and an optional command:

```
* * * * * command_to_execute
| | | | |
| | | | └─ Day of the week (0 - 7, where 0 and 7 represent Sunday)
| | | └── Month (1 - 12)
| | └──── Day of the month (1 - 31)
| └────── Hour (0 - 23)
└──────── Minute (0 - 59)
```

Each field denotes a time unit, and asterisks (*) indicate any possible value within that unit. For instance:

- in the minute field means "every minute."
- 0 in the hour field means "at the 0th hour (midnight)."
- */15 in the minute field means "every 15 minutes."

Examples:

Run a script every day at 3 AM:

javascript Copy code

```
0 3 * * * /path/to/script.sh
```

Execute a command every Monday at 8 PM:

bash Copy code

```
0 20 * * 1 /path/to/command
```

Input-Output

I/O Devices: I/O devices enable the exchange of data between the computer and the external world. They allow users to input data (e.g., keyboard input) and output data (e.g., display output, printing) from the computer system.

Types of I/O Devices:

Storage Devices: Hard disk drives (HDDs), solid-state drives (SSDs), USB drives, optical drives, etc., used for data storage.

Communication Devices: Network adapters, modems, Bluetooth devices, used for network communication.

Input Devices: Keyboards, mice, touchscreens, scanners, etc., used to input data into the computer.

Output Devices: Monitors, printers, speakers, etc., used to display or output information from the computer.

Device Queue:

The device queue is a data structure used in the operating system to manage I/O requests for a specific device. When the CPU initiates an I/O operation (e.g., reading data from a hard drive), the request is placed in the device queue associated with that I/O device. The device queue manages pending I/O requests, ensuring proper ordering and handling of I/O operations while the CPU performs other tasks.

I/O Operation Workflow:

When an application or process needs to perform an I/O operation, it initiates the request through the operating system. The OS places the I/O request in the device queue associated with the relevant I/O device. The device controller (part of the I/O device) processes the request from the queue, performs the required I/O operation (read or write), and signals completion. Upon completion, the OS notifies the requesting application or process, allowing it to continue execution.

I/O Scheduling: I/O scheduling algorithms are used to optimize the ordering and processing of I/O requests in device queues. They aim to improve system performance by prioritizing and managing the sequence of I/O operations efficiently.

Device Drivers: Device drivers are software components that allow the operating system to communicate with and control hardware devices. They act as an interface between the hardware device and the operating system, enabling the OS to send commands, receive data, and manage the device's operations. Their **responsibilities** include:

- **Providing abstraction:** Device drivers present a uniform interface to the operating system, abstracting the specific hardware details and complexities of various devices.
- **Hardware interaction:** They manage interactions with hardware components, handling input/output operations, interrupt handling, and memory access.
- **Device control:** Device drivers control and configure hardware devices based on the OS's instructions and manage the device's functionality.

Types of Device Drivers:

- **Kernel Drivers:** Run in the kernel space of the operating system, directly interacting with hardware.
- **User-Space Drivers:** Operate in user-space and interact with kernel drivers to provide a user-friendly interface for device control.

Device Managers: Device managers are software modules within the operating system responsible for managing installed hardware devices and their associated drivers. They oversee the detection, installation, configuration, and removal of hardware devices and their corresponding drivers. Their tasks include:

- **Device detection:** Identifying hardware devices connected to the system and determining the appropriate drivers needed for their operation.
- **Driver management:** Ensuring proper installation, configuration, and maintenance of device drivers, including updates and resolving driver-related issues.
- **Resource allocation:** Managing hardware resources (such as IRQs, DMA channels, memory addresses) to prevent conflicts between devices and optimize system performance.
- **User Interface:** Device managers often provide a graphical user interface (GUI) for users to view and manage installed hardware devices, update drivers, and troubleshoot issues.

Device drivers and device managers work hand-in-hand to facilitate the interaction between the operating system and hardware devices. Drivers enable the OS to communicate with hardware, while device managers oversee the installation, configuration, and management of these drivers and associated hardware devices, ensuring smooth functionality and interoperability within the system.

Number System

Number systems, complement systems, and overflow play critical roles in digital computing. Here's an overview of each:

Number System Conversion:

Decimal (Base-10): The number system most commonly used by humans, using digits 0-9. Each digit's position represents a power of 10.

Binary (Base-2): The number system used in digital systems, consisting of only 0s and 1s. Each digit's position represents a power of 2.

Hexadecimal (Base-16): A number system using digits 0-9 and letters A-F to represent values from 0 to 15. Convenient for representing binary values compactly.

Conversion between number systems involves converting a value from one base to another. For instance:

Decimal to Binary: Divide the number by 2 and record the remainders.

Binary to Decimal: Multiply each digit by its corresponding power of 2 and sum the results.

Decimal to Hexadecimal: Divide the number by 16 and convert remainders to hexadecimal digits.

Hexadecimal to Decimal: Multiply each digit by its corresponding power of 16 and sum the results.

Complements:

Two's Complement: Used to represent signed integers in binary. In a two's complement system, negative numbers are represented by flipping the bits and adding 1 to the least significant bit (LSB) of the binary representation of the positive number.

One's Complement: A binary numeral system in which the complement of a number is obtained by toggling all the bits. In one's complement, negative numbers are represented by taking the one's complement of the positive number.

Overflow: Overflow occurs when the result of an arithmetic operation exceeds the maximum representable value that can be held within a given number of bits. In digital systems with fixed word sizes (limited number of bits), arithmetic operations (addition, subtraction, multiplication) may produce results that cannot be represented within the available bits, leading to overflow. Detecting overflow requires examining the sign bits and carry bits generated during arithmetic operations. Overflow can result in incorrect computations or loss of precision in digital systems.

Patent and Cyberlaw

A patent is a legal right granted by a government to an inventor, giving them exclusive control over their invention for a specific time, usually around 20 years. This protection is in exchange for disclosing the invention to the public. To get a patent, the invention must be new, non-obvious, and useful. In the United States, the power to grant patents is derived from Article I, Section 8, Clause 8 of the U.S. Constitution, often referred to as the Patent Clause. This clause empowers Congress to promote innovation and progress by securing exclusive rights to inventors for their discoveries. Patents can cover various things like products, processes, designs, or plants. They provide legal protection, allowing inventors to prevent others from using, making, or selling their invention without permission. After the patent's term expires, the invention becomes part of the public domain for anyone to use freely.

HIPAA (Health Insurance Portability and Accountability Act):

HIPAA is a federal law enacted in 1996 in the United States. Its primary objectives are to protect sensitive patient health information (PHI) and improve the efficiency and effectiveness of the healthcare system. HIPAA has two main parts: the **Privacy Rule** and the **Security Rule.**

- The Privacy Rule establishes standards for protecting individuals' medical records and personal health information, ensuring their confidentiality and proper use.
- The Security Rule sets guidelines to safeguard electronic protected health information (ePHI) by outlining security measures that covered entities, such as healthcare providers, health plans, and clearinghouses, must implement to protect this information.

HIPAA compliance is crucial in the healthcare industry to ensure patient privacy, prevent data breaches, and impose penalties for non-compliance, including significant fines.

FISMA (Federal Information Security Management Act):

FISMA is a United States federal law passed in 2002. Its primary goal is to protect federal government information, operations, and assets against cybersecurity threats. FISMA mandates federal agencies to develop, document, and implement information security programs and practices. It requires federal agencies to create and maintain risk-based information security programs that include measures to protect data, assess vulnerabilities, and respond to security incidents. FISMA establishes a framework for continuous monitoring, risk assessment, and compliance reporting to ensure the security and integrity of federal information and information systems.

Compiled and edited by
Abrar Mahmud Hasan
December 6th, 2023